

Pościg robotów

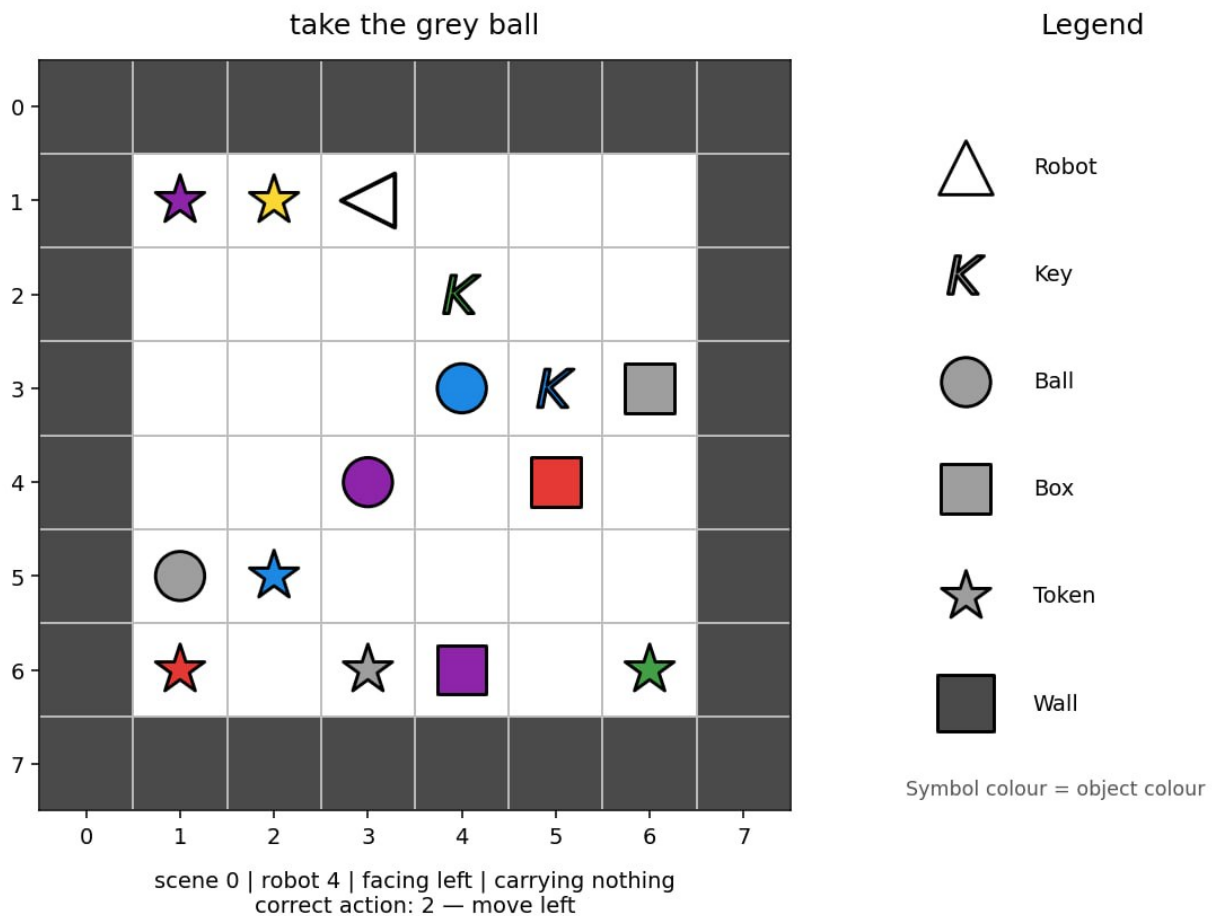
- **Limit czasu:** 5 minut
- **Środowisko:** 1 GPU (≈ 16 GB VRAM), bez dostępu do internetu
- **Rozmiar rozwiązania:** `solution.ipynb` ≤ 1 MB
- **Pamięć masowa:** 5 GB

Zadanie

Jest sześć robotów. Każdy robot działa w małym pomieszczeniu reprezentowanym na siatce (ang. grid). Każde pomieszczenie ma "obszar gry" 6×6 otoczony ścianami, więc pełna tablica `image` ma rozmiar 8×8 (obszar gry + ściany).

Każdy robot otrzymuje instrukcję w języku angielskim opisującą jego zadanie. Zrzut planszy (ang. snapshot) może zostać wykonana w dowolnym momencie podczas wykonywania zadania przez robota. Twoim celem jest przewidzenie następnej akcji robota.

Roboty nie zawsze podążają najkrótszą ścieżką. Robot 0 może zachowywać się inaczej niż Robot 1, ale każdy robot postępuje zgodnie z własnym, spójnym wzorcem. Wykorzystaj przykłady treningowe, które zawierają poprawne następne akcje, aby nauczyć się tych wzorców.



Istnieją trzy rodzaje misji:

- **pójście do** (ang. go to) obiektu, na przykład "approach the red ball";
- **podniesienie** (ang. pick up) obiektu, na przykład "grab the blue key";
- **umieszczenie jednego obiektu obok innego**, na przykład "place the red box beside the green ball".

Ta sama instrukcja może być zapisana na kilka sposobów. Zbiór testowy może zawierać nowe kombinacje znanych wyrażeń, kolorów i typów obiektów. Jednak każde słowo, wzorec wyrażenia, kolor, typ obiektu i typ misji użyte w zbiorze testowym występują również w zbiorze treningowym.

Każda próbka ma następujące pola:

Pole	Znaczenie
robot_id	którego z 6 robotów dotyczy ten przykład (0-5)
image	pomieszczenie, tablica liczb całkowitych $8 \times 8 \times 2$, w której kanał 0 zawiera wartość kategorię <code>object_idx</code> (np. 1=puste, 2=ściana, 10=robot), a kanał 1 zawiera wartość kategorię <code>colour_idx</code> (0-5).
direction	kierunek, w którym robot jest obecnie zwrócony
mission	"misja" -- widoczna instrukcja w języku naturalnym
carrying	"czy przenosi" -- null lub <code>[object_idx, colour_idx]</code> przenoszonego obiektu

Wiersze są niezależnymi snapshotami w losowej kolejności. Nie tworzą epizodów, a podczas ewaluacji nie jest dostępna żadna wcześniejsza obserwacja ani akcja.

Dostarczony `visualize_dataset.ipynb` umożliwia przeglądanie obserwacji dostępnych dla modelu w różnych sytuacjach.

Kodowanie siatki

`image[row][column] = [object_idx, colour_idx]`. Pierwszy indeks oznacza wiersz, licząc od góry do dołu, a drugi kolumnę, licząc od lewej do prawej. Tablica obejmuje zewnętrzne obramowanie ze ścian, więc wewnątrz, po którym można się poruszać, ma rozmiar 6×6 .

Identyfikatory obiektów:

id	obiekt
1	puste pole / empty cell
2	ściana / wall
5	klucz / key
6	piłka / ball
7	pudełko / box
10	robot / robot
11	żeton / token

Żetony mogą pojawiać się w pomieszczeniach, ale ich nazwy nigdy nie są użyte w misjach.

Identyfikatory kolorów to 0 red/czerwony, 1 green/zielony, 2 blue/niebieski, 3 purple/fioletowy, 4 yellow/żółty oraz 5 grey/szary. Kanał koloru nie ma znaczenia dla pustych pól i ścian.

Obraz ma wyłącznie dwa powyższe kanały. Kierunek robota jest podany jednokrotnie, w polu najwyższego poziomu `direction`; nie jest wspomniany ponownie (ang. duplicated) wewnątrz `image`.

Akcje

Dla kodów 0–3 akcje ruchu korzystają z następującego mapowania bezwzględnego:

akcja	znaczenie
0	ruch w górę
1	ruch w dół
2	ruch w lewo
3	ruch w prawo
4	podniesienie
5	upuszczenie

Pole `direction` wskazuje obecną orientację robota za pomocą następujących wartości: 0 = Góra (wiersz - 1), 1 = Dół (wiersz + 1), 2 = Lewo (kolumna - 1), 3 = Prawo (kolumna + 1).

Akcja ruchu najpierw obraca robota w danym kierunku bezwzględny, a następnie podejmuje próbę przesunięcia go o jedno pole. Ściana lub obiekt mogą zablokować ruch, ale kierunek i tak się zmienia. `pick up` i `drop` działają wyłącznie na sąsiednim polu docelowym określonym przez kierunek (np. jeśli `direction=0`, działają na `(row - 1, col)`).

Zbiór danych

Otrzymujesz dwa foldery:

Folder	Wiersze	<code>labels.json</code> ?	Zastosowanie
<code>dataset/train/</code>	60,000	dołączone	trenowanie modelu
<code>dataset/test_public/</code>	3,600	dołączone w kopii deweloperskiej	uruchomienie i samodzielna ocena pipeline

Każdy folder zawiera `observations.json`, czyli listę JSON próbek opisanych powyżej. `labels.json` jest odpowiadającą jej listą akcji (0–5).

Zbiór treningowy zawiera dokładnie po 10,000 wierszy dla każdego robota oraz po 20,000 wierszy z każdej rodziny zadań. Publiczny zbiór testowy zawiera po 600 wierszy dla każdego robota. Jeśli potrzebujesz użyć tablicy (ang. array), opakuj `image` za pomocą `numpy.asarray(...)`.

Podczas oceniania `dataset/test_public/` jest w sposób niewidoczny zastępowany ukrytym zbiorem 3,600 obserwacji w tym samym formacie, ale bez `labels.json`. Publiczna tablica wyników wykorzystuje `test_leaderboard_a`; końcowy ranking wykorzystuje `test_leaderboard_b`. Notebook, który bezwarunkowo odczytuje etykiety testowe, zakończy się niepowodzeniem. Odczytuj etykiety (labels) wyłącznie z `dataset/train/`.

Output

Zapisz `predictions.json` w katalogu roboczym notebooka. Musi to być lista JSON zawierająca po jednej całkowitoliczbowej akcji (0–5) dla każdego wiersza `dataset/test_public/observations.json`, w tej samej kolejności. Dla hipotetycznego zbioru testowego zawierającego sześć próbek poprawny wynik wyglądałby następująco:

```
[0, 3, 2, 2, 5, 4]
```

Brakujący lub niepoprawny plik JSON, niewłaściwa liczba predykcji, wartość niecałkowita albo akcja spoza `{0, 1, 2, 3, 4, 5}` zostaną odrzucone bez oceny.

Ocenianie

Wynikiem jest **średnia dokładność dla poszczególnych robotów** w skali 0–100. Dokładność jest najpierw obliczana niezależnie dla każdego robota, a następnie uśredniana dla wszystkich sześciu robotów. Każdy robot ma zatem taką samą wagę.

Sposób przesłania rozwiązania

1. Otwórz `solution.ipynb` i uruchom wszystkie komórki.
2. Potwierdź, że zapisuje `predictions.json` z 3,600 predykcjami dla publicznego zbioru testowego.
3. Jeśli chcesz, ulepsz model; dostarczony model bazowy (ang. baseline) jedynie demonstruje wymagany format danych wejściowych i wyjściowych.
4. W zakładce Git w JupyterLab *dodaj do obszaru przejściowego* (ang. stage) i zcommituj (commit) `solution.ipynb`, a następnie go wypchnij (ang. push).
5. Wróć na stronę konkursu i kliknij **Submit**.

Prześlij dokładnie jeden plik o nazwie `solution.ipynb`.